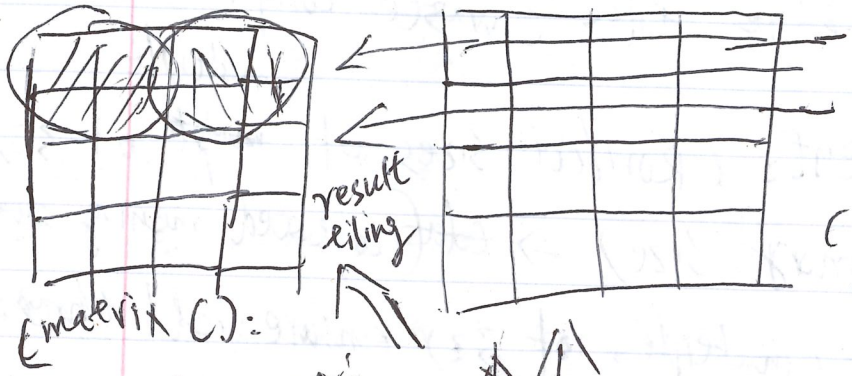
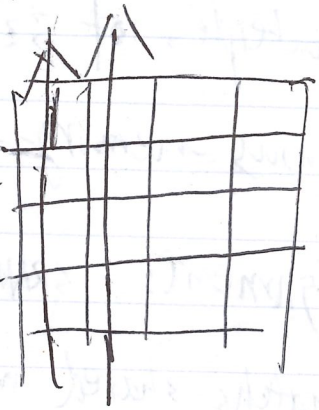
#6: memory tiling: grouping and ordering threads to minimize global memory access.

2) store and reuse information in shared memory.
shared by diff threads



cut-up the matrix into smaller 'tiles', and

- load this into shared memory for compute



non-tiled matmul: each input is read N times from global memory

tiled matmul: each input is read $\frac{N}{T}$ times from global memory, and T times each tile.

[shift mat reading from global memory to shared memory
fast

no reduction on total number of all.

- matrix elements

#1

Thread Block size alignment: cmat dims divisible by tile size (16x16)

every block covers an even portion of the matrix.

- no idel threads / no partial tiles to be handled (edge tiles (waste compute)

#2 ~~warp size~~

warp size alignment: (Row/col sizes of multiple of 32) for coalesced memory size) => ~~idel~~ (coalesced memory access)

aligned dim (multiples of 32) ensure all threads in a warp load contiguous memory → max throughput

#3 Shared memory alignment: SRAM organized by banks

=> tile (16, 32 or 64) matchs shared memory alignment boundaries every thread fetches data efficiently.

=> mis-aligned access cause serialization - e.g. when multiple threads hit the same memory ~~block~~ bank, [no bank conflicts]

#4 Tensor core tile size alignment